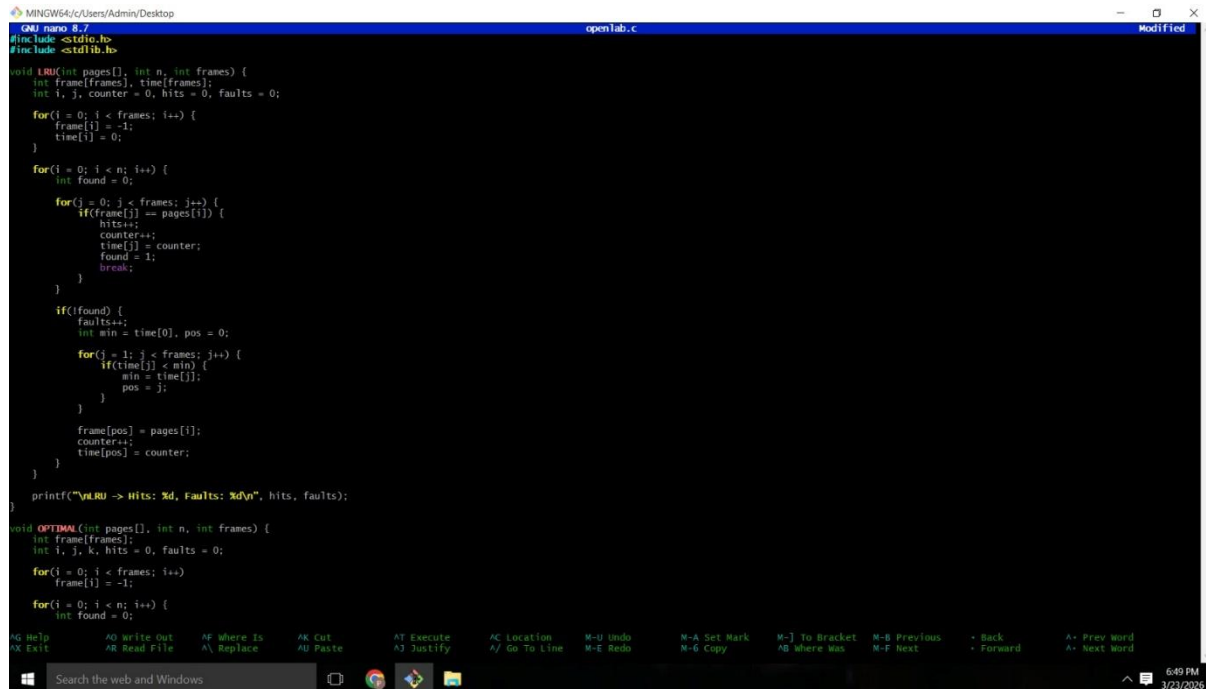


POST LAB

AIM : To implement and compare LRU (Least Recently Used) and Optimal page replacement algorithms and calculate the number of page hits, page faults, and their percentages.

Code:



```
GNU nano 8.7 openlab.c
#include <stdio.h>
#include <stdlib.h>

void LRU(int pages[], int n, int frames) {
    int frame[frames], time[frames];
    int i, j, counter = 0, hits = 0, faults = 0;

    for(i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                hits++;
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        if(!found) {
            faults++;
            int min = time[0], pos = 0;
            for(j = 1; j < frames; j++) {
                if(time[j] < min) {
                    min = time[j];
                    pos = j;
                }
            }
            frame[pos] = pages[i];
            counter++;
            time[pos] = counter;
        }
    }

    printf("LRU -> Hits: %d, Faults: %d\n", hits, faults);
}

void OPTIMAL(int pages[], int n, int frames) {
    int frame[frames];
    int i, j, k, hits = 0, faults = 0;

    for(i = 0; i < frames; i++)
        frame[i] = -1;

    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                hits++;
                found = 1;
                break;
            }
        }

        if(!found) {
            faults++;
            int index = -1, farthest = i;

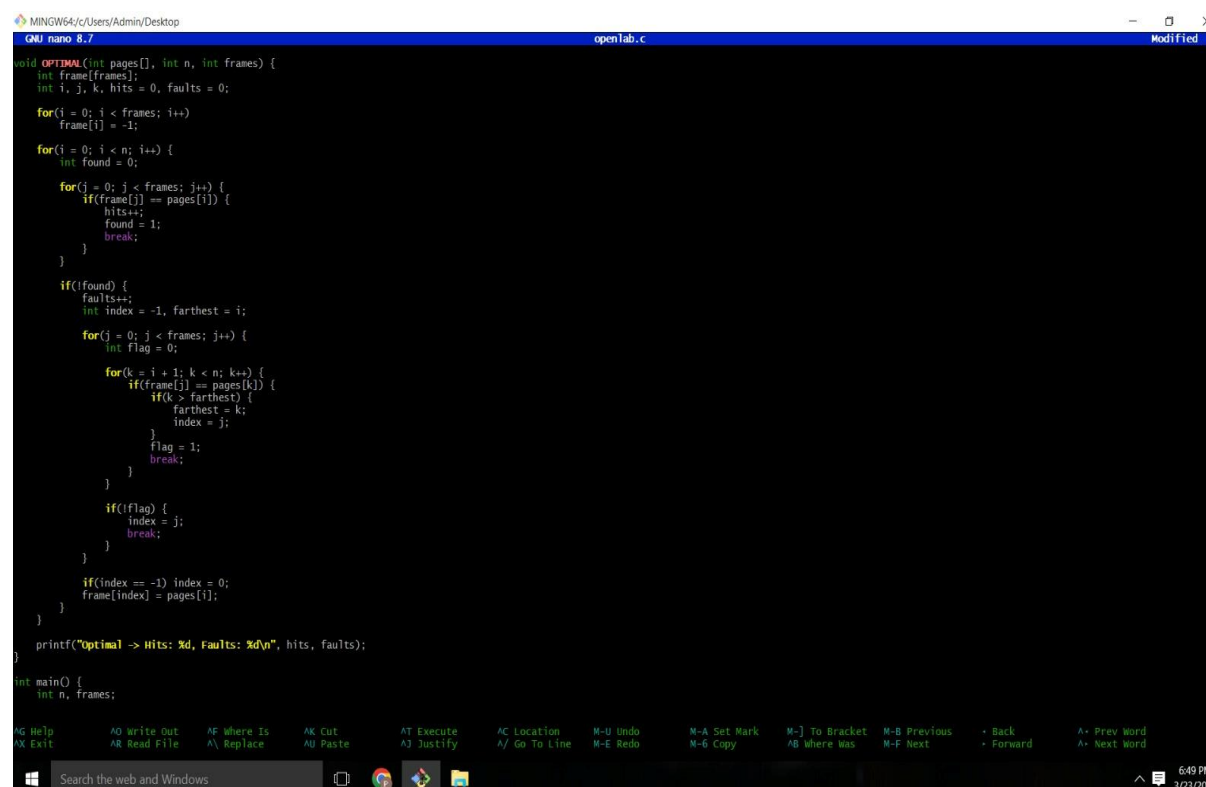
            for(j = 0; j < frames; j++) {
                if(flag = 0; {
                    for(k = i + 1; k < n; k++) {
                        if(frame[j] == pages[k]) {
                            if(k > farthest) {
                                farthest = k;
                                index = j;
                            }
                        }
                    }
                    flag = 1;
                    break;
                }
            }

            if(!flag) {
                index = j;
                break;
            }
        }

        if(index == -1) index = 0;
        frame[index] = pages[i];
    }

    printf("Optimal -> Hits: %d, Faults: %d\n", hits, faults);
}

int main() {
    int n, frames;
```



```
GNU nano 8.7 openlab.c
void OPTIMAL(int pages[], int n, int frames) {
    int frame[frames];
    int i, j, k, hits = 0, faults = 0;

    for(i = 0; i < frames; i++)
        frame[i] = -1;

    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                hits++;
                found = 1;
                break;
            }
        }

        if(!found) {
            faults++;
            int index = -1, farthest = i;

            for(j = 0; j < frames; j++) {
                if(flag = 0; {
                    for(k = i + 1; k < n; k++) {
                        if(frame[j] == pages[k]) {
                            if(k > farthest) {
                                farthest = k;
                                index = j;
                            }
                        }
                    }
                    flag = 1;
                    break;
                }
            }

            if(!flag) {
                index = j;
                break;
            }
        }

        if(index == -1) index = 0;
        frame[index] = pages[i];
    }

    printf("Optimal -> Hits: %d, Faults: %d\n", hits, faults);
}

int main() {
    int n, frames;
```

```
GNU nano 8.7 openlab.c
int index = -1, farthest = 1;
for(j = 0; j < frames; j++) {
    int flag = 0;
    for(k = j + 1; k < n; k++) {
        if(frame[j] == pages[k]) {
            if(k > farthest) {
                farthest = k;
                index = j;
            }
            flag = 1;
            break;
        }
    }
    if(!flag) {
        index = j;
        break;
    }
}
if(index == -1) index = 0;
frame[index] = pages[j];
}
printf("Optimal -> Hits: %d, Faults: %d\n", hits, faults);
}

int main() {
    int n, frames;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("\n--- RESULT ---\n");
    LRU(pages, n, frames);
    OPTIMAL(pages, n, frames);
    return 0;
}
```

AG Help
AX Exit
AO Write Out
AR Read File
AF Where Is
A\ Replace
AK Cut
AP Paste
AT Execute
AJ Justify
AC Location
A/ Go To Line
M-U Undo
M-E Redo
M-A Set Mark
M-G Copy
M-] To Bracket
AB Where Was
M-B Previous
M-P Next
• Back
• Forward
A+ Prev Word
A+ Next Word

Search the web and Windows 6:49 PM 3/23/2026

Output:

```
MINGW64/c/Users/Admin/Desktop
Admin@DESKTOP-M6MDE7A MINGW64 ~
$ cd ~/Desktop
Admin@DESKTOP-M6MDE7A MINGW64 ~/Desktop
$ nano openlab.c
Admin@DESKTOP-M6MDE7A MINGW64 ~/Desktop
$ gcc openlab.c -o openlab
Admin@DESKTOP-M6MDE7A MINGW64 ~/Desktop
$ ./openlab.exe
```

C:\Users\Admin\Desktop\openlab.exe

Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

--- LRU Algorithm ---
Total Page Faults = 10
Total Page Hits = 2
Hit Percentage = 16.67%
Fault Percentage = 83.33%

--- Optimal Algorithm ---
Total Page Faults = 7
Total Page Hits = 5
Hit Percentage = 41.67%
Fault Percentage = 58.33%

Press Enter to exit...



Search the web and Windows



